

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN
HỆ THỐNG PHÂN TÁN

Đề tài: Phát triển hệ thống chat ngang hàng P2P
(Peer-to-Peer Chat System)

Giảng viên:	Kim Ngọc Bách
Nhóm lớp:	03
Nhóm bài tập:	02
Nguyễn Minh Khánh	B22DCCN448
Bùi Thế Mạnh	B22DCCN520
Nguyễn Hữu Hưng	B22DCCN412

Hà Nội - 2026

Mục lục

Lời cảm ơn	4
CHƯƠNG I. TỔNG QUAN ĐỀ TÀI	5
1.1. Giới thiệu đề tài	5
1.2. Lý do chọn đề tài	6
1.3. Mục tiêu của đề tài	6
1.4. Phạm vi đề tài	6
CHƯƠNG II. CƠ SỞ LÝ THUYẾT	7
2.1. Tổng quan về hệ thống phân tán	7
2.2. Kiến trúc Peer-to-Peer (P2P)	7
2.2.1. Khái niệm mô hình P2P	7
2.2.2. Các loại mô hình P2P	8
2.3. Giao tiếp mạng trong hệ thống phân tán	8
2.4. Peer Discovery	9
2.4.1. Khái niệm	9
2.4.2. Bootstrap Server	9
2.5. Đồng bộ trạng thái và quản lý liveness	10
2.5.1. Heartbeat	10
2.5.2. Timeout Detection	10
2.6. Độ tin cậy truyền tin	10
2.6.1. ACK	10
2.6.2. Retry Mechanism	11
2.6.3. Message ID	11
2.7. Bảo mật truyền thông	11
2.7.1. Mã hóa RSA	11
2.7.2. Mã hóa AES	11
2.8. Mô hình thông điệp phân tán	12
2.9. Chat nhóm và tính nhất quán cuối cùng	12
2.10. Trade-offs trong hệ thống phân tán	13
CHƯƠNG III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	13
3.1. Kiến trúc tổng quan hệ thống	13
3.1.1. Peer Node	14
3.1.2. Bootstrap Server	16
3.1.3. Đặc điểm của kiến trúc hệ thống	18
3.2. Luồng hoạt động của hệ thống	18
3.3. Mô hình triển khai hệ thống	20

3.4. Thiết kế cơ sở dữ liệu	20
3.4.1. Sơ đồ quan hệ thực thể (ER Diagram)	21
3.4.2. Mô tả chi tiết từng bảng	21
3.5. Thiết kế Giao thức và các Luồng xử lý chính	24
3.5.1. Giao thức truyền tin	24
3.5.2. Luồng đăng ký và khám phá mạng(Peer Discovery)	25
3.5.3. Luồng đăng ký và khám phá mạng (Peer Discovery)	25
3.5.4. Luồng Trao đổi khóa bảo mật (Key Exchange)	27
3.5.5. Luồng Chat 1-1 và Cơ chế xác nhận (ACK / Retry)	29
Cơ chế ACK và Retry	29
CHƯƠNG IV : CÀI ĐẶT VÀ TRIỂN KHAI	30
4.1. Công nghệ sử dụng	30
4.2. Cấu trúc mã nguồn	32
4.3. Cấu hình môi trường và cơ sở dữ liệu	33
4.3.1. Chi tiết các biến môi trường cấu hình	33
CHƯƠNG V : KẾT QUẢ VÀ KIỂM THỬ	36
5.1. Giao diện Bootstrap Launcher	36
5.2. Giao diện Peer Web UI	36
5.3. Demo gửi tin nhắn trực tiếp	37
5.4. Demo gửi tin nhắn nhóm	38
5.5. Tổng hợp kết quả kiểm thử	39
CHƯƠNG VI : KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	41
1. Kết luận	41
2. Hướng phát triển trong tương lai	42
TÀI LIỆU THAM KHẢO	43

Lời cảm ơn

Nhóm chúng em xin chân thành cảm ơn Thầy Kim Ngọc Bách đã tận tình giảng dạy, hướng dẫn và hỗ trợ nhóm trong suốt quá trình học tập cũng như thực hiện bài tập lớn cuối kỳ của môn học. Những kiến thức chuyên môn, kinh nghiệm thực tiễn cùng các góp ý quý báu của Thầy đã giúp nhóm có thêm nhiều hiểu biết và hoàn thành đề tài một cách tốt nhất.

Thông qua bài tập lớn này, nhóm không chỉ được củng cố kiến thức đã học mà còn rèn luyện thêm kỹ năng làm việc nhóm, tư duy phân tích và khả năng vận dụng lý thuyết vào thực tế. Đây sẽ là những kinh nghiệm quý báu giúp chúng em trong quá trình học tập và phát triển sau này.

Mặc dù nhóm đã cố gắng hoàn thiện bài làm, tuy nhiên do kiến thức và kinh nghiệm còn hạn chế nên khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được những nhận xét và góp ý từ Thầy để bài báo cáo được hoàn thiện hơn.

Cuối cùng, nhóm xin kính chúc Thầy Kim Ngọc Bách luôn dồi dào sức khỏe, nhiều niềm vui và thành công trong công tác giảng dạy.

CHƯƠNG I. TỔNG QUAN ĐỀ TÀI

1.1. Giới thiệu đề tài

Trong bối cảnh công nghệ mạng và truyền thông phát triển mạnh mẽ, nhu cầu trao đổi thông tin trực tuyến giữa người dùng ngày càng gia tăng. Các ứng dụng nhắn tin hiện nay phần lớn hoạt động theo mô hình client-server, trong đó mọi dữ liệu đều phải đi qua máy chủ trung tâm. Mặc dù mô hình này dễ quản lý và triển khai, nhưng lại tồn tại nhiều hạn chế như phụ thuộc vào server trung tâm, dễ xảy ra quá tải, chi phí vận hành cao và nguy cơ xuất hiện điểm lỗi duy nhất (single point of failure).

Để khắc phục những hạn chế đó, mô hình mạng ngang hàng (Peer-to-Peer – P2P) đã được nghiên cứu và ứng dụng rộng rãi trong nhiều hệ thống phân tán hiện đại. Trong mô hình này, mỗi nút mạng (peer) vừa đóng vai trò là client vừa đóng vai trò là server, cho phép các peer giao tiếp trực tiếp với nhau mà không cần phụ thuộc hoàn toàn vào máy chủ trung tâm.

Xuất phát từ ý tưởng trên, nhóm thực hiện đề tài: **“Phát triển hệ thống chat ngang hàng P2P (Peer-to-Peer Chat System)”** nhằm xây dựng một ứng dụng chat cho phép nhiều người dùng trao đổi tin nhắn trực tiếp thông qua mạng P2P. Hệ thống hỗ trợ các chức năng cơ bản như tham gia mạng, khám phá peer, chat trực tiếp, chat nhóm và quản lý trạng thái online/offline của người dùng.

Thông qua đề tài, nhóm mong muốn áp dụng các kiến thức đã học trong môn Các hệ thống phân tán như giao tiếp mạng, quản lý tiến trình phân tán, xử lý đồng thời, truyền thông giữa các node và cơ chế chịu lỗi cơ bản.

1.2. Lý do chọn đề tài

Các hệ thống chat trực tuyến là một trong những ứng dụng phổ biến nhất của hệ thống phân tán. Việc xây dựng một hệ thống chat P2P giúp sinh viên hiểu rõ hơn về cách các node trong mạng giao tiếp và phối hợp với nhau mà không cần phụ thuộc hoàn toàn vào server trung tâm.

Ngoài ra, đề tài còn mang tính thực tiễn cao bởi mô hình P2P hiện đang được sử dụng trong nhiều lĩnh vực như chia sẻ file, livestream, blockchain và các ứng dụng truyền thông thời gian thực.

Thông qua việc triển khai hệ thống, nhóm có cơ hội:

- Nắm vững nguyên lý hoạt động của mạng ngang hàng.

- Tìm hiểu cơ chế truyền thông bằng TCP Socket.
- Thực hành xử lý đa luồng và nhiều kết nối đồng thời.
- Hiểu rõ cách quản lý trạng thái phân tán trong hệ thống.
- Rèn luyện kỹ năng thiết kế và phát triển phần mềm mạng.

1.3. Mục tiêu của đề tài

Đề tài hướng tới việc xây dựng một hệ thống chat ngang hàng hoạt động ổn định và đáp ứng các yêu cầu cơ bản của một hệ thống phân tán.

Các mục tiêu chính gồm:

- Xây dựng hệ thống chat theo mô hình Peer-to-Peer.
- Cho phép các peer tham gia và rời mạng linh hoạt.
- Hỗ trợ gửi và nhận tin nhắn trực tiếp giữa các peer.
- Hỗ trợ chat nhóm giữa nhiều peer.
- Xây dựng cơ chế peer discovery để khám phá các peer trong mạng.
- Hiện thị trạng thái online/offline của người dùng.
- Đảm bảo truyền tin đáng tin cậy và xử lý lỗi kết nối cơ bản.
- Áp dụng kỹ thuật đa luồng để xử lý nhiều kết nối cùng lúc.

1.4. Phạm vi đề tài

Trong phạm vi bài tập lớn môn học, hệ thống tập trung triển khai các chức năng cốt lõi của một hệ thống chat P2P bao gồm:

- Kết nối peer thông qua TCP Socket.
- Gửi và nhận tin nhắn trực tiếp.
- Quản lý danh sách peer online.
- Chat nhóm cơ bản.
- Peer discovery thông qua bootstrap server.
- Xử lý mất kết nối đơn giản.

Đề tài chưa tập trung vào các vấn đề chuyên sâu như:

- Bảo mật nâng cao.
- Đồng bộ dữ liệu phức tạp.
- Hệ thống phân tán quy mô lớn.
- Tối ưu hiệu năng mạng.
- Cơ chế consensus hoặc replication nâng cao.

CHƯƠNG II. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về hệ thống phân tán

Hệ thống phân tán (Distributed System) là tập hợp nhiều máy tính độc lập kết nối với nhau thông qua mạng và phối hợp hoạt động như một hệ thống thống nhất. Các thành phần trong hệ thống có khả năng trao đổi dữ liệu, chia sẻ tài nguyên và phối hợp xử lý nhằm đáp ứng các yêu cầu của người dùng.

Đặc điểm chính của hệ thống phân tán gồm:

- Nhiều nút hoạt động đồng thời.
- Các thành phần giao tiếp qua mạng.
- Không có bộ nhớ dùng chung.
- Có khả năng mở rộng và chịu lỗi.
- Dữ liệu và trạng thái được phân tán trên nhiều node.

Hệ thống chat P2P là một ví dụ điển hình của hệ thống phân tán, trong đó các peer giao tiếp trực tiếp với nhau thay vì phụ thuộc hoàn toàn vào máy chủ trung tâm.

2.2. Kiến trúc Peer-to-Peer (P2P)

2.2.1. Khái niệm mô hình P2P

Peer-to-Peer (P2P) là mô hình mạng trong đó mỗi node vừa đóng vai trò client vừa đóng vai trò server. Các peer có thể trực tiếp trao đổi dữ liệu với nhau mà không cần thông qua server trung gian cho toàn bộ quá trình truyền thông.

Trong hệ thống chat P2P:

- mỗi peer có thể gửi tin nhắn,
- nhận tin nhắn,
- duy trì kết nối với các peer khác.

Khác với mô hình client-server truyền thống, mô hình P2P giúp:

- giảm tải cho server trung tâm,
- tăng khả năng mở rộng,
- tránh điểm lỗi duy nhất (Single Point of Failure).

Tuy nhiên, mô hình này cũng làm tăng độ phức tạp trong:

- quản lý kết nối,
- phát hiện peer,
- xử lý lỗi mạng,

- đồng bộ trạng thái

2.2.2. Các loại mô hình P2P

a) Pure P2P

Không có server trung tâm. Mọi peer đều bình đẳng.

Ví dụ:

- BitTorrent (một phần),
- blockchain networks.

b) Hybrid P2P

Có sử dụng một server hỗ trợ discovery hoặc quản lý metadata, nhưng dữ liệu chính vẫn truyền trực tiếp giữa các peer.

Hệ thống của đề tài sử dụng mô hình **Hybrid P2P**, trong đó:

- bootstrap server hỗ trợ peer discovery,
- dữ liệu chat truyền trực tiếp giữa các peer.

2.3. Giao tiếp mạng trong hệ thống phân tán

Các node trong hệ thống phân tán cần giao tiếp thông qua mạng bằng các giao thức truyền thông.

Hệ thống sử dụng:

- WebSocket/TCP Socket để thiết lập kết nối hai chiều giữa các peer.

Ưu điểm:

- truyền dữ liệu thời gian thực,
- hỗ trợ giao tiếp song công (full-duplex),
- độ trễ thấp.

Mỗi peer có thể:

- mở kết nối tới peer khác,
- đồng thời lắng nghe kết nối đến.

Điều này giúp mỗi peer vừa hoạt động như client vừa hoạt động như server

2.4. Peer Discovery

2.4.1. Khái niệm

Peer discovery là cơ chế giúp các peer tìm thấy nhau trong mạng phân tán.

Trong hệ thống P2P, khi một peer mới tham gia mạng, nó cần biết:

- địa chỉ IP,
- cổng kết nối,
- danh sách peer đang hoạt động.

Nếu không có peer discovery, các node sẽ không thể thiết lập kết nối trực tiếp.

2.4.2. Bootstrap Server

Đề tài sử dụng bootstrap server để:

- đăng ký peer mới,
- lưu danh sách peer online,
- phân phối thông tin peer,
- phát hiện peer rời mạng.

Quy trình hoạt động:

1. Peer gửi yêu cầu đăng ký tới bootstrap server.
2. Server lưu thông tin peer.
3. Server trả về danh sách peer hiện có.
4. Peer thiết lập kết nối trực tiếp với các peer khác.

Cách tiếp cận này giúp:

- đơn giản hóa discovery,
- giảm độ phức tạp của hệ thống.

Tuy nhiên, bootstrap server vẫn là một điểm tập trung và có thể trở thành điểm lỗi nếu server ngừng hoạt động

2.5. Đồng bộ trạng thái và quản lý liveness

Trong hệ thống phân tán, cần xác định peer nào còn hoạt động và peer nào đã mất kết nối.

Hệ thống sử dụng:

- heartbeat,
- timeout detection.

2.5.1. Heartbeat

Heartbeat là gói tin được gửi định kỳ từ peer tới bootstrap server để thông báo rằng peer vẫn đang hoạt động.

2.5.2. Timeout Detection

Nếu server không nhận heartbeat trong một khoảng thời gian nhất định:

- peer được xem là offline,
- danh sách peer sẽ được cập nhật.

Cơ chế này giúp:

- duy trì trạng thái online/offline chính xác,
- giảm kết nối lỗi,
- hỗ trợ tính chịu lỗi cơ bản.

2.6. Độ tin cậy truyền tin

Trong môi trường mạng phân tán, việc mất gói tin hoặc mất kết nối có thể xảy ra.

Để đảm bảo độ tin cậy, hệ thống sử dụng:

- ACK (Acknowledgement),
- retry mechanism,
- message ID.

2.6.1. ACK

Sau khi nhận tin nhắn:

- peer nhận gửi ACK về peer gửi,
- xác nhận rằng tin nhắn đã được nhận thành công.

2.6.2. Retry Mechanism

Nếu không nhận ACK sau một khoảng thời gian:

- hệ thống sẽ gửi lại tin nhắn,
- retry được giới hạn số lần để tránh flooding.

2.6.3. Message ID

Mỗi tin nhắn có một định danh duy nhất (UUID):

- hỗ trợ phát hiện trùng lặp,
- tránh xử lý cùng một tin nhắn nhiều lần,
- hỗ trợ idempotence.

2.7. Bảo mật truyền thông

Trong hệ thống phân tán, dữ liệu truyền qua mạng có nguy cơ bị nghe lén hoặc giả mạo.

Đề tài sử dụng mô hình mã hóa lai (Hybrid Encryption):

- RSA dùng để trao đổi khóa,
- AES dùng để mã hóa nội dung tin nhắn.

2.7.1. Mã hóa RSA

RSA là thuật toán mã hóa bất đối xứng:

- public key dùng để mã hóa,
- private key dùng để giải mã.

RSA được sử dụng để:

- trao đổi khóa AES an toàn.

2.7.2. Mã hóa AES

AES là thuật toán mã hóa đối xứng có tốc độ xử lý nhanh và phù hợp cho dữ liệu lớn.

Sau khi trao đổi khóa:

- các peer sử dụng AES để mã hóa nội dung chat.

Giải pháp hybrid crypto giúp:

- đảm bảo bảo mật,
- tăng hiệu năng truyền thông.

2.8. Mô hình thông điệp phân tán

Các thông điệp trong hệ thống được tổ chức theo cấu trúc thống nhất gồm:

- loại thông điệp (type),
- peer gửi,
- peer nhận,
- timestamp,

- message ID,
 - payload.
- Ví dụ:
- CHAT
 - GROUP_CHAT
 - ACK
 - HEARTBEAT
 - PUBLIC_KEY_EXCHANGE

Thiết kế này giúp:

- dễ mở rộng hệ thống,
- hỗ trợ xử lý phân tán,
- dễ kiểm tra và debug.

2.9. Chat nhóm và tính nhất quán cuối cùng

Trong chức năng chat nhóm:

một tin nhắn được gửi tới nhiều peer, trạng thái hệ thống được cập nhật phân tán.

Do độ trễ mạng và trạng thái phân tán:

các peer có thể nhận cập nhật ở thời điểm khác nhau.

Hệ thống áp dụng mô hình: eventual consistency.

Điều này nghĩa là: dữ liệu có thể tạm thời chưa đồng bộ, nhưng cuối cùng sẽ đạt trạng thái nhất quán.

2.10. Trade-offs trong hệ thống phân tán

Hệ thống P2P tồn tại nhiều đánh đổi giữa:

- hiệu năng,
- độ phức tạp,
- tính mở rộng,
- tính chịu lỗi.

Ưu điểm

- Giảm phụ thuộc server trung tâm.

- Tăng khả năng mở rộng.
- Truyền dữ liệu trực tiếp nhanh hơn.

Nhược điểm

- Khó quản lý kết nối.
- Phức tạp trong NAT traversal.
- Khó đồng bộ trạng thái.
- Xử lý lỗi phức tạp hơn client-server.

Đề tài sử dụng mô hình Hybrid P2P nhằm cân bằng giữa:

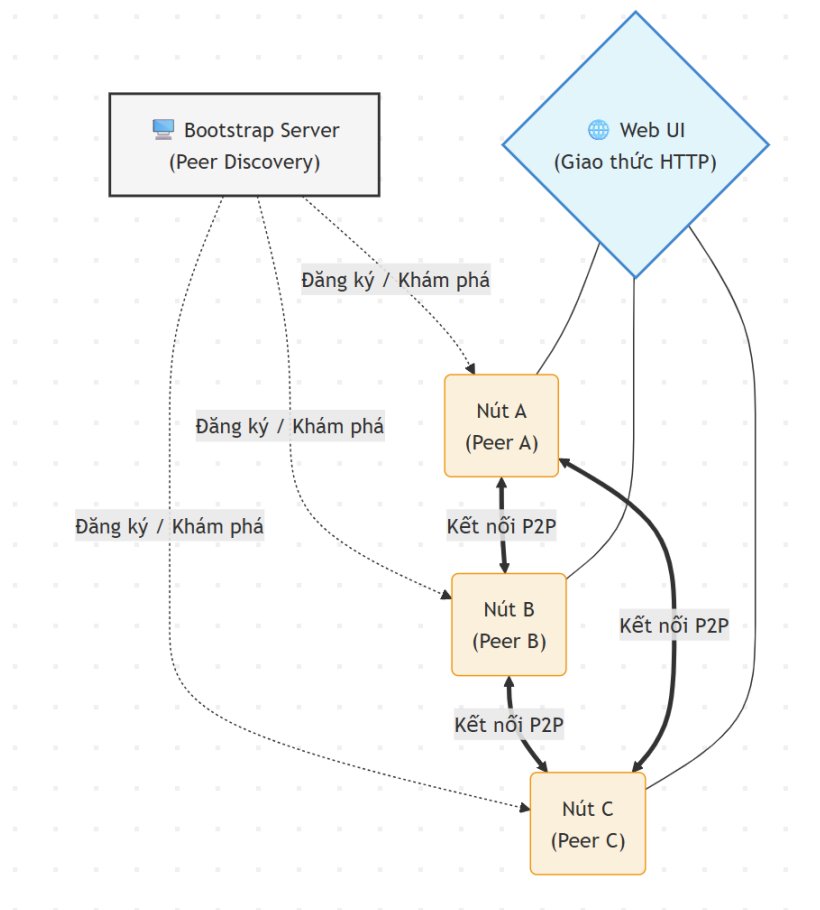
- tính phân tán,
- tính đơn giản trong triển khai,
- khả năng quản lý hệ thống.

CHƯƠNG III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng quan hệ thống

Hệ thống chat được xây dựng theo mô hình **Hybrid Peer-to-Peer (Hybrid P2P)**, trong đó các peer giao tiếp trực tiếp với nhau để trao đổi dữ liệu, đồng thời sử dụng một bootstrap server nhẹ để hỗ trợ quản lý và khám phá peer trong mạng.

Khác với mô hình client-server truyền thống, trong kiến trúc P2P mỗi peer vừa đóng vai trò là client vừa đóng vai trò là server. Điều này cho phép các node trong mạng có thể trực tiếp gửi và nhận dữ liệu mà không cần truyền toàn bộ thông tin qua một máy chủ trung tâm.



Sơ đồ kiến trúc tổng quan hệ thống P2P

Kiến trúc tổng thể của hệ thống gồm hai thành phần chính:

3.1.1. Peer Node

Peer Node là thành phần trung tâm của hệ thống, đại diện cho mỗi người dùng tham gia mạng chat. Mỗi peer hoạt động độc lập và có khả năng giao tiếp trực tiếp với các peer khác thông qua kết nối mạng.

Một peer đảm nhận đồng thời hai vai trò:

- **Client**
 - Thiết lập kết nối tới các peer khác.
 - Gửi tin nhắn trực tiếp hoặc tin nhắn nhóm.
 - Gửi heartbeat tới bootstrap server.
- **Server**
 - Lắng nghe các kết nối đến từ peer khác.
 - Nhận và xử lý thông điệp.
 - Phản hồi ACK để xác nhận đã nhận dữ liệu.

Các chức năng chính của Peer Node gồm:

a) Gửi và nhận tin nhắn

Peer có thể:

- gửi tin nhắn trực tiếp tới một peer khác,
- gửi tin nhắn nhóm tới nhiều peer,
- nhận và hiển thị tin nhắn theo thời gian thực.

Dữ liệu được truyền trực tiếp giữa các peer mà không cần đi qua bootstrap server.

b) Quản lý kết nối mạng

Mỗi peer duy trì danh sách các peer đang kết nối và quản lý trạng thái của các kết nối đó.

Peer có khả năng:

- mở kết nối mới,
- đóng kết nối lỗi,
- tự động reconnect trong một số trường hợp mất kết nối.

Hệ thống sử dụng cơ chế đa luồng (multithreading/asynchronous handling) để cho phép:

- gửi dữ liệu,
- nhận dữ liệu,
- xử lý nhiều kết nối đồng thời.

c) Mã hóa và bảo mật dữ liệu

Trước khi gửi tin nhắn:

- dữ liệu sẽ được mã hóa bằng AES,
- khóa AES được trao đổi thông qua RSA.

Cơ chế hybrid encryption này giúp:

- tăng tính bảo mật,
- giảm nguy cơ nghe lén dữ liệu trên mạng,
- tối ưu hiệu năng truyền tải.

d) Đảm bảo độ tin cậy truyền tin

Mỗi thông điệp được gắn:

- message_id,
- timestamp,
- thông tin người gửi và người nhận.

Sau khi nhận tin nhắn:

- peer nhận gửi ACK xác nhận,

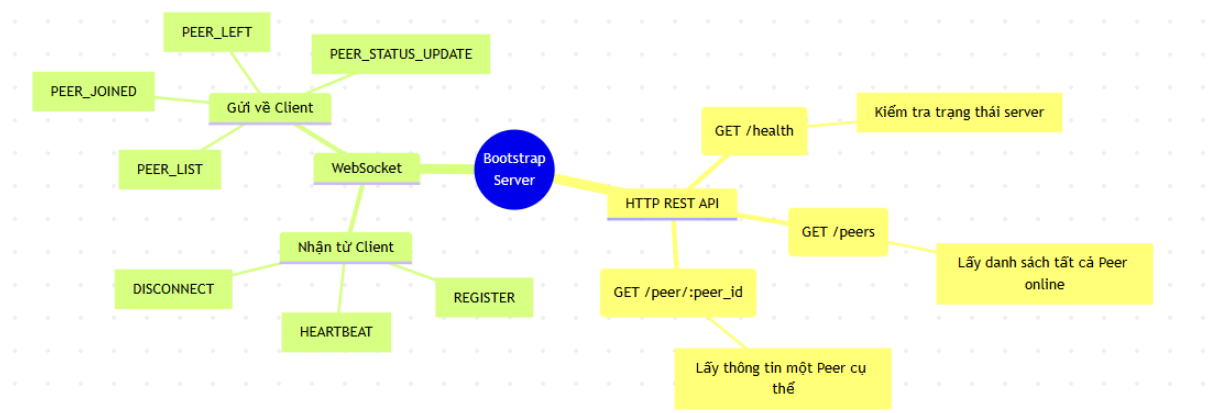
- peer gửi sẽ retry nếu không nhận ACK trong thời gian quy định.
- Cơ chế này giúp:
- giảm mất dữ liệu,
 - tránh xử lý trùng lặp,
 - đảm bảo truyền tin đáng tin cậy.



Bản đồ API của Peer Node

3.1.2. Bootstrap Server

Bootstrap Server là thành phần hỗ trợ hệ thống hoạt động hiệu quả hơn trong môi trường phân tán. Đây là một server nhẹ, không tham gia xử lý nội dung chat mà chỉ đóng vai trò hỗ trợ peer discovery và quản lý trạng thái mạng.



Bản đồ API của Bootstrap Server

Các chức năng chính của Bootstrap Server gồm:

a) Quản lý danh sách peer online

Khi một peer tham gia mạng:

- peer sẽ gửi thông tin đăng ký tới bootstrap server,
- server lưu địa chỉ IP, cổng kết nối và trạng thái hoạt động của peer.

Danh sách peer online được cập nhật liên tục dựa trên:

- heartbeat,
- timeout detection.

b) Hỗ trợ peer discovery

Khi peer mới tham gia:

- bootstrap server trả về danh sách các peer hiện đang hoạt động,
- peer mới sử dụng thông tin này để thiết lập kết nối trực tiếp.

Điều này giúp:

- đơn giản hóa quá trình khám phá peer,
- giảm thời gian thiết lập mạng,
- hỗ trợ mở rộng hệ thống.

c) Theo dõi trạng thái hoạt động của peer

Bootstrap server định kỳ nhận heartbeat từ các peer.

Nếu một peer:

- không gửi heartbeat trong khoảng thời gian xác định,
- hoặc mất kết nối,

server sẽ:

- đánh dấu peer offline,
- loại peer khỏi danh sách hoạt động,
- broadcast trạng thái mới tới các peer khác.

d) Đồng bộ thông tin mạng

Khi có peer:

- tham gia,
- rời mạng,
- hoặc thay đổi trạng thái,

bootstrap server sẽ phát tán thông tin cập nhật tới các peer liên quan nhằm duy trì tính nhất quán của hệ thống.

3.1.3. Đặc điểm của kiến trúc hệ thống

Kiến trúc của hệ thống có các đặc điểm chính sau:

a) Phân tán

Dữ liệu không phụ thuộc hoàn toàn vào server trung tâm mà được trao đổi trực tiếp giữa các peer.

b) Khả năng mở rộng

Số lượng peer có thể tăng mà không gây áp lực lớn lên bootstrap server vì dữ liệu chat không đi qua server trung tâm.

c) Tính chịu lỗi cơ bản

Nếu một peer mất kết nối:

- các peer khác vẫn tiếp tục hoạt động,
- hệ thống cập nhật trạng thái online/offline tương ứng.

d) Giao tiếp thời gian thực

Các peer sử dụng WebSocket/TCP Socket để truyền dữ liệu hai chiều với độ trễ thấp.

3.2. Luồng hoạt động của hệ thống

Quá trình hoạt động của hệ thống diễn ra theo các bước sau:

Bước 1: Peer khởi tạo và đăng ký với Bootstrap Server

Khi người dùng khởi động ứng dụng:

- peer tạo socket lắng nghe,
- gửi yêu cầu đăng ký tới bootstrap server.

Thông tin đăng ký bao gồm:

- peer ID,
- địa chỉ IP,
- cổng kết nối,
- public key.

Bootstrap server lưu thông tin này vào danh sách peer online.

Bước 2: Nhận danh sách peer đang hoạt động

Sau khi đăng ký thành công:

- bootstrap server trả về danh sách các peer hiện đang online.

Danh sách này giúp peer:

- biết các node hiện có trong mạng,
- thiết lập kết nối trực tiếp.

Bước 3: Thiết lập kết nối trực tiếp giữa các peer

Peer sử dụng thông tin nhận được để:

- mở kết nối WebSocket/TCP tới các peer khác,

- bắt đầu quá trình handshake.
- Trong quá trình này:
- các peer trao đổi public key RSA,
- thiết lập shared AES key cho phiên truyền thông.

Bước 4: Trao đổi tin nhắn trực tiếp

Sau khi kết nối thành công:

- các peer có thể gửi và nhận tin nhắn trực tiếp.

Quy trình gửi tin gồm:

1. Tạo message object.
2. Gắn message_id và timestamp.
3. Mã hóa nội dung bằng AES.
4. Gửi dữ liệu qua socket.
5. Chờ ACK xác nhận.

Bước 5: Xử lý ACK và retry

Sau khi nhận dữ liệu:

- peer nhận gửi ACK xác nhận.

Nếu peer gửi:

- không nhận ACK trong thời gian timeout,
- hệ thống sẽ retry gửi lại tin nhắn.

Cơ chế này giúp đảm bảo:

- độ tin cậy truyền tin,
- hạn chế mất dữ liệu.

Bước 6: Duy trì trạng thái hoạt động

Trong quá trình hoạt động:

- peer định kỳ gửi heartbeat tới bootstrap server.

Nếu peer mất kết nối:

- bootstrap server phát hiện timeout,
- cập nhật trạng thái offline,
- thông báo cho các peer còn lại.

3.3. Mô hình triển khai hệ thống

Hệ thống được triển khai theo mô hình mạng phân tán gồm:

- một bootstrap server,
- nhiều peer node kết nối qua mạng LAN hoặc Internet.

Mỗi peer có thể:

- hoạt động độc lập,
- tham gia hoặc rời mạng bất kỳ lúc nào,
- giao tiếp trực tiếp với nhiều peer đồng thời.

Mô hình này phù hợp với:

- các ứng dụng chat phân tán,
- hệ thống cộng tác thời gian thực,
- các ứng dụng mạng ngang hàng quy mô nhỏ và trung bình.

3.4. Thiết kế cơ sở dữ liệu

Hệ thống dùng cơ sở dữ liệu cố định (Database như MySQL, SQL Server hay MongoDB). Toàn bộ dữ liệu được lưu trên bộ nhớ tạm (RAM) để tạo nên đặc điểm thiết kế của hệ thống p2p ẩn danh, bảo mật

3.4.1. Sơ đồ quan hệ thực thể (ER Diagram)



Sơ đồ quan hệ thực thể ERD

3.4.2. Mô tả chi tiết từng bảng

Bảng BOOTSTRAP_SERVER

Bảng/thực thể BOOTSTRAP_SERVER đại diện cho máy chủ bootstrap dùng để hỗ trợ peer discovery và quản lý danh sách các peer đang hoạt động trong mạng.

Bootstrap server không tham gia trực tiếp vào quá trình truyền nội dung chat mà chỉ đóng vai trò:

- quản lý thông tin peer,
- hỗ trợ peer mới tham gia mạng,
- đồng bộ trạng thái online/offline

Các thuộc tính

Thuộc tính	Kiểu dữ liệu	Mô tả
port	int	Cổng dịch vụ của bootstrap server
peers	Map	Cấu trúc lưu danh sách các peer đang hoạt động

Bảng PEER_INFO

PEER_INFO dùng để lưu trữ thông tin mô tả của từng peer trong mạng. Đây là thực thể trung gian giúp bootstrap server quản lý các node đang hoạt động.

Mỗi peer khi đăng ký vào mạng sẽ tạo một bản ghi PEER_INFO.

Các thuộc tính

Thuộc tính	Kiểu dữ liệu	Mô tả
peer_id	string	Định danh duy nhất của peer
ip	string	Địa chỉ IP của peer
port	int	Cổng dịch vụ của peer

public_key	String	Khóa công khai RSA của peer
status	String	Trạng thái hoạt động (ONLINE/OFFLINE)
ui_active	boolean	Trạng thái giao diện người dùng

Bảng PEER

Thực thể PEER đại diện cho một node hoạt động thực tế trong hệ thống P2P.

Mỗi peer:

- vừa đóng vai trò client,
- vừa đóng vai trò server.

Peer có khả năng:

- gửi dữ liệu,
- nhận dữ liệu,
- quản lý kết nối mạng,
- xử lý mã hóa và giải mã.

Các thuộc tính

Thuộc tính	Kiểu dữ liệu	Mô tả
peerId	string	Định danh của peer
port	int	Cổng dịch vụ WebSocket/HTTP

publicKey	string	Khóa công khai RSA
privateKey	string	Khóa bí mật RSA
status	string	Trạng thái hoạt động

Bảng MESSAGE

MESSAGE là thực thể quan trọng nhất trong hệ thống, đại diện cho dữ liệu trao đổi giữa các peer.

Mỗi thông điệp trong hệ thống đều được biểu diễn dưới dạng một object có cấu trúc thống nhất.

Các thuộc tính

Thuộc tính	Kiểu dữ liệu	Mô tả
message_id	string	UUID duy nhất của tin nhắn
type	string	Loại thông điệp
from	string	ID peer gửi
to	string	ID peer nhận
timestamp	long	Thời gian gửi
payload	json	Nội dung dữ liệu

3.5. Thiết kế Giao thức và các Luồng xử lý chính

Bảng nguồn	Bảng đích	Loại quan hệ	Mô tả
BOOTSTRAP_SERVER	PEER_INFO	1 — N	Một bootstrap server quản lý nhiều peer.
PEER	MESSAGE	1 — N	Một peer có thể gửi và nhận nhiều tin nhắn.
PEER_INFO	PEER	1 — 1	PEER_INFO chứa thông tin mô tả của một peer thực tế trong mạng

Trong hệ thống phân tán P2P, việc thiết kế giao thức giao tiếp là yếu tố sống còn để các nút (peer) có thể hiểu và làm việc với nhau mà không cần sự can thiệp liên tục của server trung tâm. Hệ thống sử dụng giao thức dựa trên TCP với định dạng dữ liệu JSON.

3.5.1. Giao thức truyền tin

Trong hệ thống chat P2P, giao thức truyền tin đóng vai trò quan trọng trong việc đảm bảo các peer có thể:

- giao tiếp trực tiếp với nhau,
- truyền dữ liệu thời gian thực,
- đồng bộ trạng thái mạng,
- đảm bảo tính tin cậy và bảo mật của dữ liệu.

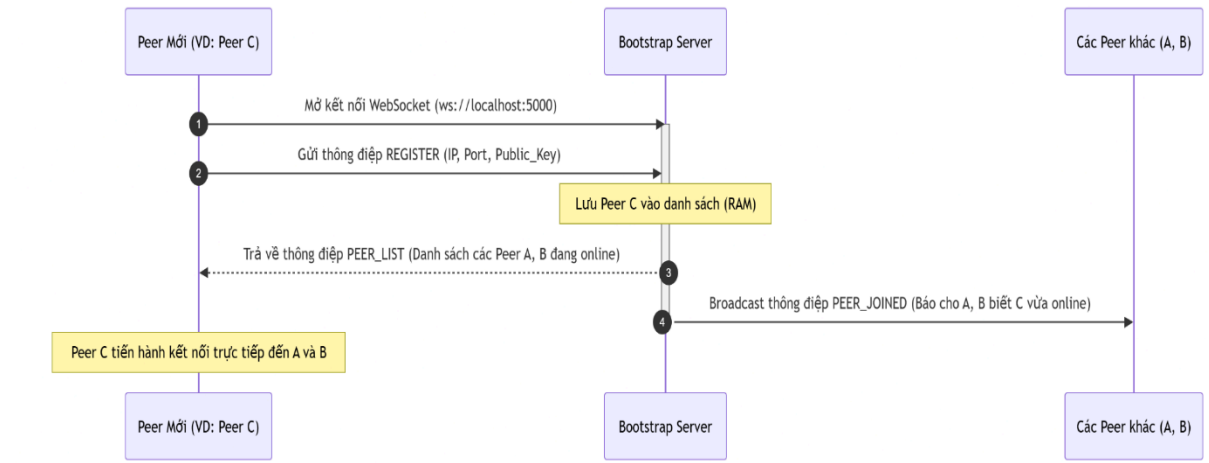
Hệ thống sử dụng:

- TCP Socket hoặc WebSocket làm nền tảng truyền thông giữa các peer.

Các giao thức này cung cấp:

- kết nối hai chiều (full-duplex),
- truyền dữ liệu ổn định,
- độ trễ thấp,
- phù hợp với ứng dụng chat thời gian thực.

3.5.2. Luồng đăng ký và khám phá mạng(Peer Discovery)



3.5.3. Luồng đăng ký và khám phá mạng (Peer Discovery)

Trong hệ thống chat P2P, cơ chế Peer Discovery giúp các peer mới tham gia có thể tìm và kết nối với các peer đang hoạt động trong mạng. Hệ thống sử dụng một Bootstrap Server để hỗ trợ quá trình này.

Quy trình hoạt động diễn ra như sau:

Bước 1: Peer kết nối tới Bootstrap Server

Khi khởi động, peer mới sẽ mở kết nối WebSocket tới Bootstrap Server thông qua địa chỉ và cổng đã cấu hình.

Ví dụ:

ws://localhost:5000

Bước 2: Gửi yêu cầu đăng ký

Peer gửi thông điệp REGISTER chứa:

- Peer ID
- Địa chỉ IP
- Port
- Public Key RSA

Ví dụ:

```
{
  "type": "REGISTER",
  "peer_id": "Peer_C",
  "port": 6001,
  "public_key": "RSA_PUBLIC_KEY"
}
```

Bước 3: Bootstrap Server lưu thông tin peer

Sau khi nhận yêu cầu:

- Bootstrap Server lưu thông tin peer vào danh sách các node đang online trong bộ nhớ RAM.

Thông tin lưu gồm:

- Peer ID
- IP
- Port
- Public Key
- Trạng thái hoạt động

Bước 4: Trả về danh sách peer đang online

Bootstrap Server gửi lại thông điệp PEER_LIST chứa danh sách các peer hiện đang hoạt động trong mạng.

Ví dụ:

```
{  
  "type": "PEER_LIST",  
  "peers": ["Peer_A", "Peer_B"]  
}
```

Bước 5: Thiết lập kết nối trực tiếp

Peer mới sử dụng danh sách nhận được để:

- kết nối trực tiếp tới các peer khác,
- bắt đầu trao đổi dữ liệu P2P.

Từ thời điểm này:

- dữ liệu chat sẽ truyền trực tiếp giữa các peer,
- không đi qua Bootstrap Server.

Bước 6: Broadcast trạng thái peer mới

Bootstrap Server gửi thông điệp PEER_JOINED tới các peer khác để thông báo có peer mới tham gia mạng.

Điều này giúp:

- đồng bộ trạng thái hệ thống,
- cập nhật danh sách peer online theo thời gian thực.

Ý nghĩa của cơ chế Peer Discovery

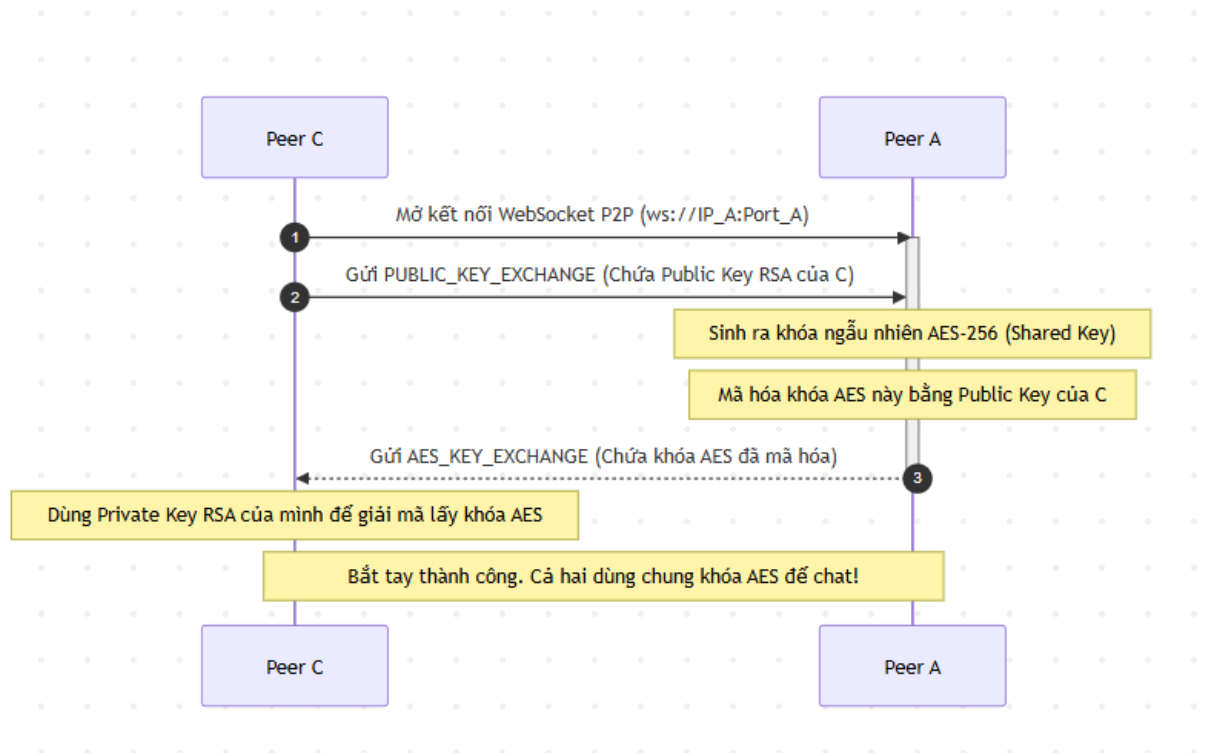
Cơ chế Peer Discovery giúp:

- các peer dễ dàng tìm thấy nhau trong mạng,
- giảm độ phức tạp kết nối P2P,

- hỗ trợ mở rộng hệ thống,
- duy trì trạng thái online/offline của các node trong môi trường phân tán.

3.5.4. Luồng Trao đổi khóa bảo mật (Key Exchange)

xảy ra ngay sau khi peer c phát hiện ra peer a. cả 2 sẽ bắt tay để thống nhất khoá bảo mật(hybrid Encryption)



Tổng quan trước khi bắt đầu

Trước khi quá trình này diễn ra, Peer C đã tự tạo cho mình một cặp khóa RSA bao gồm:

- **Public Key (Khóa công khai):** Có thể chia sẻ cho bất kỳ ai.
- **Private Key (Khóa bí mật):** Chỉ Peer C giữ lại và tuyệt đối không chia sẻ.

Các bước chi tiết trong luồng trao đổi

Bước 1: Thiết lập kết nối

- **Hành động:** Peer C chủ động mở một kết nối WebSocket dạng Peer-to-Peer (P2P) tới Peer A.
- **Địa chỉ:** Kết nối được thực hiện thông qua địa chỉ IP và Port của Peer A (ví dụ: ws://IP_A:Port_A).

Bước 2: Gửi Public Key (Khóa công khai)

- **Hành động:** Ngay sau khi kết nối WebSocket được mở, Peer C gửi một thông điệp có tên là `PUBLIC_KEY_EXCHANGE` đến Peer A.
- **Nội dung:** Thông điệp này chứa Public Key RSA của Peer C. Mục đích là để Peer A dùng khóa này mã hóa thông tin mật ở bước sau.

Bước 3: Xử lý tại Peer A (Sinh khóa và Mã hóa) Sau khi nhận được Public Key của Peer C, Peer A thực hiện hai việc quan trọng ở hệ thống của mình:

1. **Sinh khóa AES:** Peer A tự động tạo ra một khóa mã hóa đối xứng ngẫu nhiên là AES-256 (Shared Key). Đây sẽ là chìa khóa chính dùng để mã hóa và giải mã nội dung tin nhắn chat sau này.
2. **Mã hóa khóa AES:** Để gửi khóa AES này cho Peer C một cách an toàn trên môi trường mạng (tránh bị kẻ gian nghe lén), Peer A sử dụng chính Public Key RSA của Peer C vừa nhận được để mã hóa khóa AES. Chỉ có Private Key của Peer C mới có thể giải mã được gói tin này.

Bước 4: Trả khóa AES đã mã hóa về cho Peer C

- **Hành động:** Peer A gửi một thông điệp phản hồi `AES_KEY_EXCHANGE` lại cho Peer C.
- **Nội dung:** Thông điệp này chứa khóa AES-256 đã bị mã hóa.

Bước 5: Xử lý tại Peer C (Giải mã khóa AES)

- **Hành động:** Nhận được thông điệp từ Peer A, Peer C sử dụng Private Key RSA (khóa bí mật mà chỉ C có) để giải mã gói tin.
- **Kết quả:** Sau khi giải mã thành công, Peer C thu được khóa AES-256 (Shared Key) nguyên bản mà Peer A đã tạo ra ở Bước 3.

Kết quả (Bắt tay thành công)

Lúc này, quá trình "Handshake" (Bắt tay) đã hoàn tất. Cả Peer C và Peer A hiện tại đều đang nắm giữ chung một khóa AES-256.

Từ thời điểm này trở đi, hai bên không dùng RSA nữa mà sẽ sử dụng chung khóa AES-256 này để mã hóa và giải mã nội dung tin nhắn chat với nhau.

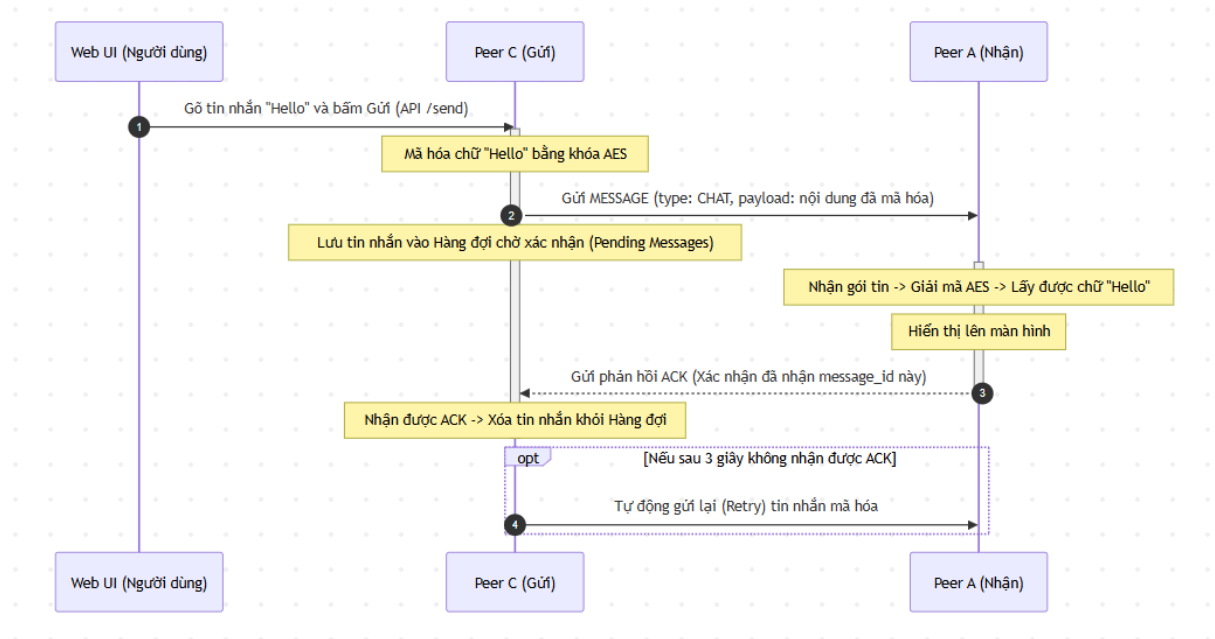
Đặc điểm:

🔒 **Bảo mật cao (nhờ RSA):** Khóa AES dùng để chat được truyền đi một cách an toàn, không ai bắt gói tin ở giữa có thể đọc được vì họ không có Private Key của C.

🏢 **Hiệu suất tốt (nhờ AES):** Việc mã hóa/giải mã lượng dữ liệu lớn (như tin nhắn, file) bằng AES nhanh và nhẹ hơn rất nhiều so với dùng RSA.

3.5.5. Luồng Chat 1-1 và Cơ chế xác nhận (ACK / Retry)

Xảy ra khi người dùng gõ tin nhắn và gửi đi. Bao gồm cả cơ chế đảm bảo tin không bị rớt mạng.



Cơ chế ACK và Retry

Để đảm bảo độ tin cậy trong quá trình truyền tin giữa các peer, hệ thống sử dụng cơ chế **ACK (Acknowledgement)** và **Retry**.

ACK (Acknowledgement)

Sau khi nhận và xử lý thành công tin nhắn, peer nhận sẽ gửi một thông điệp ACK về peer gửi để xác nhận rằng dữ liệu đã được nhận thành công.

Mỗi tin nhắn đều có `message_id` duy nhất nhằm:

- xác định chính xác tin nhắn cần xác nhận,
- tránh xử lý trùng lặp dữ liệu.

Ví dụ ACK:

```
{
  "type": "ACK",
```

```
"message_id": "msg_001"
}
```

Retry Mechanism

Sau khi gửi tin nhắn:

- hệ thống lưu message vào hàng đợi chờ xác nhận (Pending Queue),
- bắt đầu bộ đếm timeout.

Nếu sau một khoảng thời gian (ví dụ 3 giây) không nhận được ACK:

- hệ thống sẽ tự động gửi lại tin nhắn.

Mỗi message chỉ được retry một số lần giới hạn nhằm:

- tránh flooding mạng,
- giảm tải hệ thống.

Ý nghĩa của cơ chế ACK và Retry

Cơ chế ACK và Retry giúp:

- đảm bảo tin nhắn được truyền thành công,
- giảm nguy cơ mất dữ liệu,
- tăng độ tin cậy của hệ thống chat P2P,
- hỗ trợ xử lý lỗi mạng và mất kết nối tạm thời trong môi trường phân tán.

CHƯƠNG IV : CÀI ĐẶT VÀ TRIỂN KHAI

4.1. Công nghệ sử dụng

Thành phần / Tính năng	Công nghệ lựa chọn	Vai trò trong hệ thống Chat P2P
Backend (Xử lý cốt lõi)	NodeJS / Python	Xử lý logic mạng (Socket Programming) , thiết lập máy chủ lắng nghe (Listener) trên mỗi máy khách để nhận và quản lý các

		kết nối P2P trực tiếp.
Giao thức (Protocol)	TCP / WebSocket	Tạo kênh kết nối hai chiều liên tục, thời gian thực (Real-time). WebSocket là lựa chọn tối ưu khi giao diện chạy trên nền Web, còn TCP thường dùng cho ứng dụng Desktop thuần.
Frontend (Giao diện)	Web (HTML, JS)	Xây dựng giao diện người dùng (UI), tiếp nhận thao tác nhập/gửi tin nhắn và hiển thị luồng trò chuyện lên màn hình.
Xử lý đồng thời (Concurrency)	Thread (Luồng) / Async (Bất đồng bộ)	Đảm bảo ứng dụng hoạt động mượt mà không bị "đơ" chặn (Non-blocking). Giúp hệ thống vừa có thể lắng nghe tin nhắn đến, vừa xử lý thao tác gõ phím của người dùng cùng lúc.

		(Lưu ý: NodeJS mặc định dùng Async/Event Loop, Python có thể dùng Thread hoặc Asyncio).
Bảo mật (Encryption)	RSA và AES cơ bản	Mã hóa đầu cuối bảo vệ nội dung chat. Kết hợp cả hai: Dùng RSA để thực hiện quá trình trao đổi khóa an toàn (Key Exchange), sau đó dùng chung khóa AES để mã hóa toàn bộ tin nhắn nhằm tối ưu tốc độ xử lý.

4.2. Cấu trúc mã nguồn

ChatP2P/

```

├── src/
│   ├── bootstrap-server/
│   │   └── server.js      # Bootstrap server for peer discovery
│   ├── peer-node/
│   │   ├── peer.js       # Peer node implementation
│   │   └── test-peers.js  # Test peer instances
│   ├── common/
│   │   └── constants.js   # Global constants

```



```

| | | └─ encryption.js    # RSA + AES module
| | | └─ utils.js        # Utility functions
| | └─ ui/
| |   └─ index.html      # Main web interface on port 8080
| |   └─ server.js       # Main web server
| |   └─ peer-ui-server.js # Legacy redirect to port 8080
| └─ package.json
| └─ start-all.js       # Script to start all services
| └─ bussiness_plan.md   # Business plan
| └─ technical_plan.md   # Technical plan

```

4.3. Cấu hình môi trường và cơ sở dữ liệu

Hệ thống sử dụng tệp cấu hình biến môi trường `.env` nằm ở thư mục gốc để quản lý toàn bộ các thông số liên quan đến mạng, cơ sở dữ liệu và bảo mật.

4.3.1. Chi tiết các biến môi trường cấu hình

Khi khởi chạy hệ thống, file `.env` sẽ được nạp qua thư viện `dotenv` để cấu hình các thông số sau:

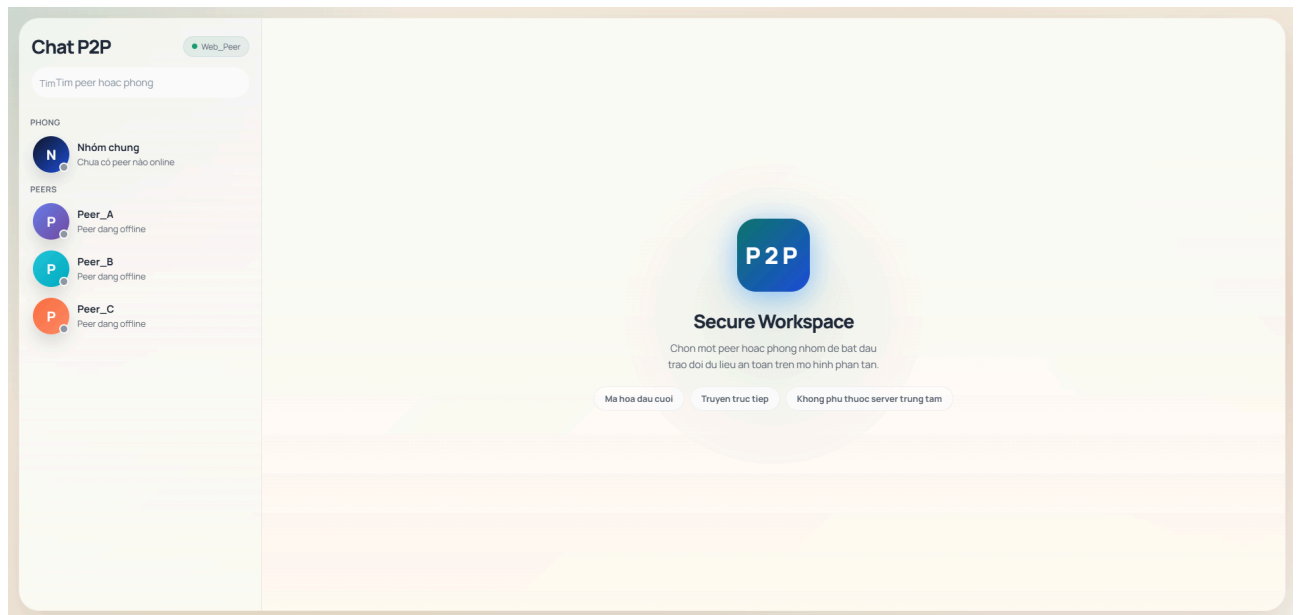
Mục (Cấu hình)	Giá trị mặc định	Biến môi trường	Giải thích chi tiết
Node.js	>= 14.0.0	-	Phiên bản môi trường Node.js tối thiểu bắt buộc để chạy hệ thống.
Bootstrap host	localhost	BOOTSTRAP_HOST	Địa chỉ IP hoặc tên miền của máy chủ Bootstrap (dùng để các Peer tìm thấy nhau).
Bootstrap port	5000	BOOTSTRAP_PORT	Cổng mạng (Port) mà Bootstrap Server mở ra để lắng nghe kết nối.

Bootstrap URL	ws://localhost:5000	BOOTSTRAP_URL	Đường dẫn WebSocket đầy đủ để các Peer kết nối trực tiếp vào Bootstrap Server.
Bootstrap heartbeat timeout	30000 ms	-	Thời gian chờ tối đa. Nếu sau 30s không nhận được tín hiệu sống, server sẽ xóa Peer đó khỏi danh sách online.
Peer heartbeat interval	10000 ms	-	Tần suất (10s/lần) mà các Peer tự động gửi tín hiệu "Tôi vẫn online" (ping) để duy trì kết nối mạng.
Peer retry limit	3 lần	-	Số lần tối đa tự động gửi lại tin nhắn nếu người nhận chưa xác nhận đã nhận (chưa có ACK).
Peer retry timeout	5000 ms	-	Thời gian tối đa chờ phản hồi (ACK) sau mỗi lần gửi tin. Quá 5s sẽ gửi lại lần tiếp theo.
Web UI host	localhost	UI_HOST	Địa chỉ IP chạy giao diện web của ứng dụng chat cho người dùng.
Web UI port	8080	UI_PORT	Cổng mạng (Port) để truy cập giao diện web trên trình duyệt.
RSA key size	2048	-	Độ dài khóa bất đối xứng RSA (2048-bit), chuẩn an toàn phổ

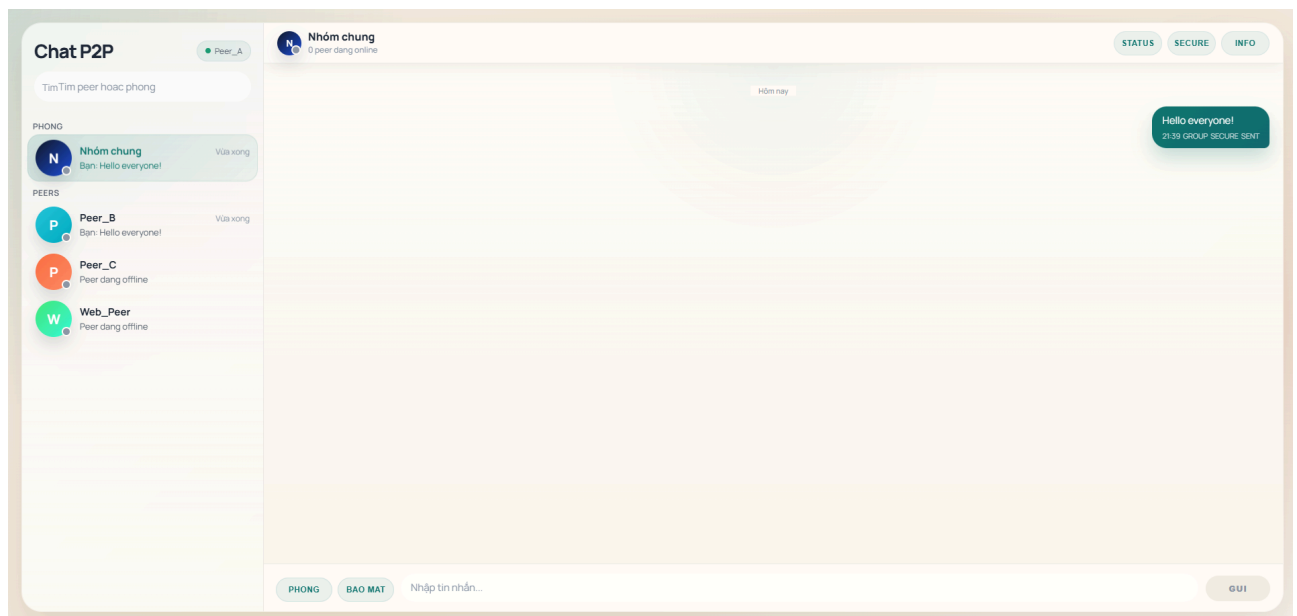
			biến hiện nay dùng để trao đổi khóa AES.
AES algorithm	aes-256-cbc	-	Chuẩn thuật toán mã hóa đối xứng tốc độ cao dùng để mã hóa bảo mật toàn bộ nội dung tin nhắn.
Salt rounds	10	-	Số vòng lặp thêm chuỗi ngẫu nhiên (salt) khi băm dữ liệu, giúp chống lại các cuộc tấn công dò mật khẩu.
Log level	info	LOG_LEVEL	Mức độ chi tiết của nhật ký hệ thống được in ra (info = hiển thị thông tin chung và lỗi, bỏ qua chi tiết debug).
Log format	json	-	Định dạng lưu nhật ký dưới dạng JSON, giúp các hệ thống theo dõi dễ dàng đọc và phân tích tự động.
Test peer ports	3001, 3002, 3003	-	Các cổng mạng cấp sẵn để bạn mở cùng lúc 3 Peer mô phỏng trên cùng một máy tính khi test code.
Test peer IDs	Peer_A, Peer_B, P	-	Tên định danh (ID) mặc định tự gán cho các máy Peer khi chạy trong môi trường thử nghiệm (Test).

CHƯƠNG V : KẾT QUẢ VÀ KIỂM THỬ

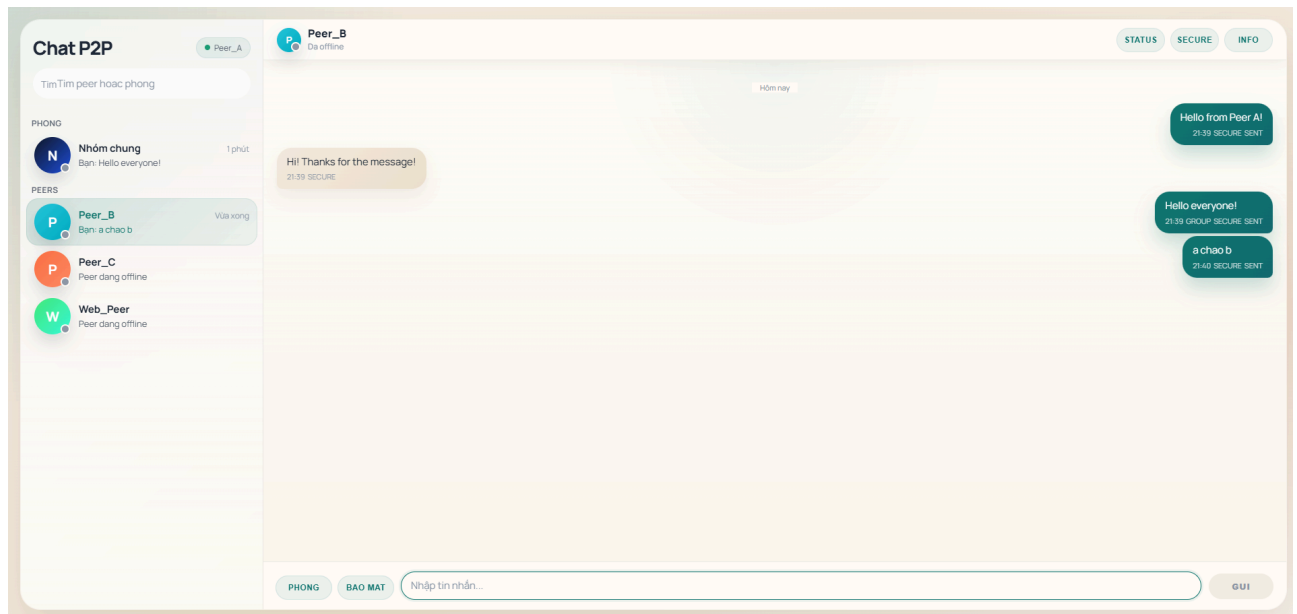
5.1. Giao diện Bootstrap Launcher



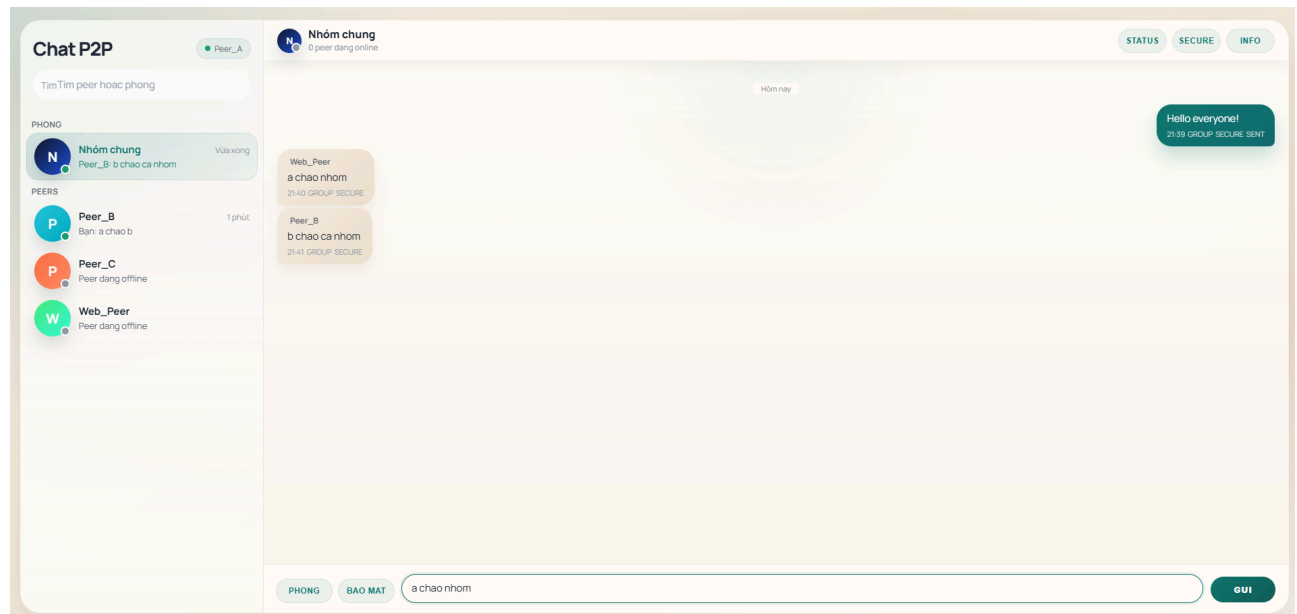
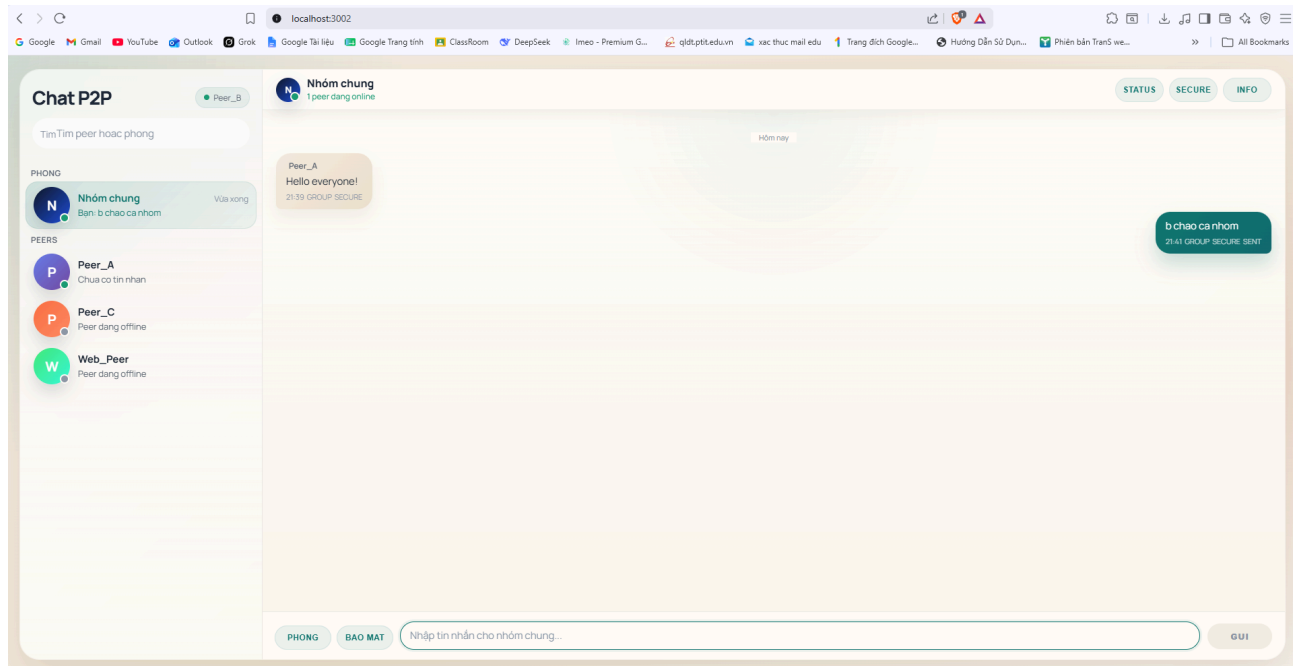
5.2. Giao diện Peer Web UI



5.3. Demo gửi tin nhắn trực tiếp



5.4. Demo gửi tin nhắn nhóm



5.5. Tổng hợp kết quả kiểm thử

Hạng mục kiểm thử	Trạng thái	Đánh giá chi tiết
Mục tiêu cốt lõi (Mô hình P2P)	Đạt	Xây dựng thành công mô hình ngang hàng. Bootstrap server chỉ dùng để đăng ký/khám phá, không xử lý tập trung.
Tham gia mạng P2P	Đạt	Peer đăng ký thành công với bootstrap server và nhận được danh sách.
Chat trực tiếp (1-1)	Đạt	Tin nhắn được truyền trực tiếp qua kết nối WebSocket giữa các peer.
Chat nhóm (Group Chat)	Đạt	Hỗ trợ gửi tin hiệu broadcast đến nhiều peer trong mạng.
Khám phá Peer (Discovery)	Đạt	Bootstrap server lưu trữ chính xác thông tin peer online và trả về danh sách hợp lệ.
Quản lý trạng thái Online/Offline	Đạt	Có heartbeat 10 giây/lần. Tự động đánh dấu offline sau 30 giây không nhận được tín hiệu.
Truyền tin đáng tin cậy	Đạt	Hoạt động tốt cơ chế xác nhận (ACK) + retry tối đa 3 lần. Có xử lý lỗi khi mất kết nối.

Kỹ thuật & Xử lý đồng thời	Đạt	Sử dụng WebSocket (TCP). Ứng dụng Node.js event-driven cho phép gửi/nhận đồng thời nhiều kết nối.
Bảo mật (Encryption)	Đạt (Nâng cao)	Có mã hóa an toàn với RSA-2048 (trao đổi khóa) và AES-256 (dữ liệu tin nhắn).
Giao diện người dùng (Web UI)	Đạt một phần	Đã có UI cơ bản kèm kiến trúc rõ ràng, nhưng cần tích hợp chặt chẽ hơn với WebSocket thực tế.
Lưu trữ & chuyển tiếp (Store-and-forward)	Chưa có	Chưa xử lý việc lưu tạm tin nhắn khi gửi cho peer đang offline để chuyển tiếp sau.
Truyền tải tập tin (File Transfer)	Chưa có	Hệ thống hiện tại chưa hỗ trợ chức năng gửi/nhận file.
Mô phỏng mạng động (Churn)	Chưa có	Chưa có kịch bản test mô phỏng tình trạng các peer vào/ra mạng liên tục.
Lưu trữ lịch sử chat	Chưa có	Chưa có cơ sở dữ liệu lưu trữ lịch sử tin nhắn.

CHƯƠNG VI : KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết luận

Đồ án "Hệ thống Chat Ngang Hàng Bảo Mật (P2P Secure Chat)" đã giải quyết thành công bài toán xây dựng một mạng truyền thông phi tập trung, đảm bảo tính riêng tư cho người dùng. Thông qua quá trình nghiên cứu và triển khai, nhóm đã đạt được các kết quả nổi bật sau:

Về kiến trúc: Xây dựng thành công mô hình mạng P2P kết hợp Bootstrap Server (Tracker), giúp các node (người dùng) có thể tự động khám phá mạng, tra cứu địa chỉ và thiết lập kết nối WebSocket trực tiếp với nhau mà không cần đi qua máy chủ trung gian.

Về bảo mật: Tích hợp thành công cơ chế mã hóa lai (Hybrid Encryption) kết hợp giữa RSA (trao đổi khóa) và AES-256 (mã hóa dữ liệu). Điều này đảm bảo tính chất mã hóa đầu cuối (End-to-End Encryption), ngăn chặn rủi ro nghe lén ngay cả khi gói tin bị đánh cắp trên đường truyền mạng.

Về tính sẵn sàng: Hiện thực hóa thành công cơ chế "Nhịp tim" (Heartbeat) và cơ chế xác nhận gói tin (ACK/Retry), giúp mạng tự động phát hiện và xử lý các node bị ngắt kết nối đột ngột, đảm bảo tính chịu lỗi cơ bản của một hệ thống phân tán.

Thông qua đồ án này, nhóm đã củng cố sâu sắc kiến thức về lập trình mạng (Socket/WebSocket), nguyên lý hoạt động của hệ thống phân tán, cũng như cách áp dụng mật mã học vào thực tiễn.

2. Hướng phát triển trong tương lai

Dù hệ thống đã đáp ứng tốt các yêu cầu cơ bản, nhưng để trở thành một sản phẩm thực tế và hoàn thiện hơn, nhóm đề xuất một số hướng phát triển tiếp theo như sau:

Loại bỏ hoàn toàn điểm nút duy nhất (SPOF): Thay vì dùng một Bootstrap Server duy nhất để tìm kiếm peer, có thể áp dụng thuật toán DHT (Distributed Hash Table) như Kademlia hoặc Chord để biến mạng lưới thành P2P thuần túy 100%.

Lưu trữ và chuyển tiếp tin nhắn ngoại tuyến (Store-and-Forward): Xây dựng cơ chế lưu trữ tạm thời các thông điệp khi peer đích đang offline và tự động đẩy (push) tin nhắn khi họ online trở lại. Tích hợp cơ sở dữ liệu cục bộ (như IndexedDB hoặc SQLite) ở phía client để giữ lại lịch sử chat.

Hỗ trợ đa phương tiện và WebRTC: Nâng cấp từ WebSocket lên giao thức WebRTC để cho phép truyền tải file dung lượng lớn, gọi điện thoại (Audio) hoặc gọi video trực tiếp giữa các peer.

Định danh và xác thực người dùng: Tích hợp hệ thống định danh kỹ thuật số hoặc chữ ký số gắn với Public Key để ngăn chặn giả mạo danh tính (Spoofing/Man-in-the-Middle) khi người dùng tham gia mạng.

TÀI LIỆU THAM KHẢO

- [1] Andrew S. Tanenbaum, Maarten van Steen (2007). Distributed Systems: Principles and Paradigms (2nd Edition). Prentice Hall. (Tham khảo về kiến trúc hệ thống phân tán và mạng P2P).
- [2] Node.js Foundation. Node.js v18.x Documentation - Crypto & Child Processes. Truy cập tại: <https://nodejs.org/docs/> (Tham khảo về cách xử lý đa luồng và sử dụng module mã hóa trong Node.js).
- [3] WebSocket Protocol (RFC 6455). IETF. Truy cập tại: <https://datatracker.ietf.org/doc/html/rfc6455> (Tham khảo cơ chế truyền thông điệp liên tục, song công full-duplex).
- [4] William Stallings (2016). Cryptography and Network Security: Principles and Practice (7th Edition). Pearson. (Tham khảo thuật toán mã hóa đối xứng AES và bất đối xứng RSA).
- [5] Thư viện `ws` (WebSockets cho Node.js). Repository: <https://github.com/websockets/ws>